Weekly Goals

- Improve model training data
    - Add the road segment to training data and clean unneeded rows
    - Look into increased weather coverage
        - Look into automation through api
    - Look into reducing the amount of needed negative samples
    - Create a file to transform the ordered csv from C.R.I.S. into useable training data

Some model work can be done as well, but I want to know (as of 3/30/2026) if a model has been decided on or not. Depending on the answer (and what model) some goals to improve on the model may be added. However, for now improving training data comes from the last meeting where the team lead identified that the primary issue was bad data. I agree with this sentiment and while the TA made a good point about having users of the app be able to report crashes and retrained the AI based on these crashes, a good starting point is needed as the app will be shown off before any users can use the app.

Data Generation

The primary issue facing the project main feature, the model, is the data. Weather is an issue and most of the models want a road segment option built into them. The other issue is the size of the data. These are all issues I want to address this week.

Cleaning the data

The simplest things to start with are just cleaning up the data a bit. Last week I grabbed a new set of data with these columns.

```
"Crash ID","Average Daily Traffic Amount","City","County","Crash Date","Crash
Month","Crash Time","Crash Year","Day of Week","Hour of Day","Road Class","Rural Urban
Type","Section","Street Name","Surface Condition","Weather Condition"
```

Average Daily Traffic amount was seen not to be useful (which should be addressed further in the document) and Crash ID needs to removed from the data set which leads to this

```
City,County,Crash Date,Crash Month,Crash Time,Crash Year,Day of Week,Hour of Day,Road
Class,Rural Urban Type,Section,Street Name,Surface Condition,Weather Condition,Crash
```

With the crash flag being added during the first step of data generation. From here, the next change should be to combine the segment and the road name. Without doing this, I am worried that the AI could be taught that any segment 1 of a road is dangerous. Thus, I think it would be prudent to combine road names and segments

The current steps are:

```
    # what_to_run[0] = add crash column to modified csv and remove crash id column
    # what_to_run[1] = identify and replace non critical roads and create modified csv
    # what_to_run[2] = create road reference csv
    # what_to_run[3] = create negative samples
    # what_to_run[4] = combine crash and negative samples into training data csv
```

A new step between step 0 and 1 needs to be added to combine segment and road so we get something like

<Street Name> S

The following is pulled from the code and indicated it is IH0635 with the section being 1.

```
IH0635 S1
```

This name is not difficult to change in the case that it is unideal or roads ending with S<number> are common. At minimum, this is a good starting point. The new steps are

```
    # what_to_run[0] = add crash column to modified csv and remove crash id column
    # what_to_run[1] = combine street name and segment into single street name column
    # what_to_run[2] = identify and replace non critical roads and create modified csv
    # what_to_run[3] = create road reference csv
    # what_to_run[4] = create negative samples
    # what_to_run[5] = combine crash and negative samples into training data csv
```

The only other modification can be made inside the new step 1. That can be made to be a general cleanup step. I would like to take the Hour of Day and which C.R.i.S. formatted as:

```
01:00 - 01:59
```

I would like to turn this into a single hour such as

```
1
```

To represent the first hour. This change needs to be reflected in the create negative samples step (step 4) but otherwise should be just fine to make with minimal issues.

After this cleanup step, the columns and an example column are such

```
City,County,Crash Date,Crash Month,Crash Time,Crash Year,Day of Week,Hour of Day,Road
Class,Rural Urban Type,Street Name,Surface Condition,Weather Condition,Crash

UNIVERSITY PARK,DALLAS,2025-01-01,1,154,2025,WEDNESDAY,1,CITY STREET,No Data,SOUTHWESTERN
BLVD,1 - DRY,1 - CLEAR,1
```

Rural Urban Type is still a column that I think could be removed, but otherwise, the clean up step should be finished

Overview

Removed Crash ID
Combined Street Name and Section
Simplified Hour of Day

There are two more important things to do in regards to this draft of data generation: Increasing weather coverage and reducing the amount of redundant negative samples

Weather Coverage

At present, the weather is just taken from a single source at NOAA
https://www.ncei.noaa.gov/access/search/data-search/global-hourly

For the current test data, this is taken from DFW airport and the station located there. NOAA has several posts across many cities and while I do not want to apply all the weather observation posts, even a singular post per city inside of a county could do much for ensuring that weather is more accurately portrayed. Increasing the scope of the data in terms of both time and space will come when working with the C.R.I.S. data.

The fastest and most scalable option would be to see if there exists some sort of API that could deliver the records of NOAA upon querying. The other option would be to see if NOAA allows for ordering of information in a similar way to C.R.I.S. API would be preferable that way in the case that this project transfers to other states or locations, a new API could be plugged in.

Theoretically, manually downloading weather from NOAA is possible, but API would make the task painless and would speed up creating training data by large factors.

The solution I discovered is that there is the option to use python to bulk download files. So, a new directory was created in the csv file so the directory is done as such

```
- csv
        csv_overview.py
        csv_editing.py
        - weather
                csv_weather.py
                weather_download.py
                isd-history.csv
                - csv_data
```

Where weather_download.py needs isd-history.csv in order to run. Isd-history provides the id of different weather stations from NOAA. A few alterations to variables inside weather_download.py can fetch and download the csv files of all the weather. These are then put in csv_data

There are some issues with this system. The most prevalent one is that isd-history has some station ids that do not have downloadable data. There also seems to be stations with downloadable history that do not appear in isd-history. Once I can get a full pipeline going, I will check to see if there is a better version of isd-history available.

Right now, the generated code needs specific station ids, I want to change this so that a city name (or some type of name) can be given inside an array and isd-history is checked for any stations names matching that city name. There are some cities (like Dallas) with multiple stations, so perhaps the user could be queried to see which of x stations they would like to use. This can be implemented after trial and error.

An issue that I have found is that there is a city column but the cities are not called as they are. For example, a row for Dallas is:

```
"722580","13960","DALLAS LOVE FIELD AIRPORT",
"US","TX","KDAL","+32.838","-096.836","+0147.6","19460726","20250827"
```

Where Dallas is KDAL. This shouldn't be much of an issue, but we no longer can just type the name of a city without having the translation.

Upon reflection, isb-history is entirely airport data. This isn't bad and could be good for a first run through, but isn't the city by city information I was hoping for. It still has 572 entries for Dallas, I just now need to know if there is at least one per city.

Apparently

```
Rows with CTRY=US and STATE=TX: 490
Distinct normalized STATION NAME values ("Dallas, TX" / "Dallas TX" style merged): 464
```

This means that 464 cities are accounted for inside of TX. With only 252 counties, this should be good, but given that this only tracks airports, and only some of them, I am now worried about the feasibility of this method of weather gathering. If this does not have at least one station per county, then the information this uses becomes useless.

The issue with lining this up is that while some of the unique cities have cities lined up with them, it does not have that universally at the front and some do not have cities named at all. I still do not want to give up on this idea. So I want to see if I can use some sort of geo-location or lookup type API to check what city/county these locations are in. From there I can even use this data to line up locations with what county they are in which is a column that lives inside of the crash records.

From here cursor imported geolocation services from the census data to create a list of all the counties that were represented in the 464 stations to determine how many counties were represented by station data

```
Unique counties: 145
```

This is unfortunately 109 short of the amount of counties that exist in Texas. Thus, this data is likely unable to be used given the large margin of counties it leaves out.

However, given how big the data is already, It is possible that this will work.

I will consult with the team and see if this is acceptable. If they take 145 to be enough counties for a pass of the data for the final project, then I will move forward and actually try to implement the system from start to finish. At present, it is just a collection of functions that don't necessarily flow well together.

Team Decision

On 4/2/2026, I got a go ahead to continue. We seem to be of the mind that so long as we can get the information for the DFW area, that this is a 'good enough' area at present. In my opinion even half of Texas is a good scope of data. NOAA seems to be the most accurate, though I have done some looking into other sources. I will continue to implement this weather system, though there are potentially issues with fetching all the data needed.

At present there are multiple layers to this weather system with different things needing work

Preprocessing	Taking a csv file and transforming it into a FinalWeather.csv. This includes filtering all the columns to just what we need and producing hourly weather data and road conditions. This is finished, but needs to be modified to produce multiple files in an easily accessible location. It also needs to be an exported function that can be looped.
Downloading of files	For the NOAA system, the isd-history station includes station id information. This should take a year as well as the locations of the data we want. This is not finished. - This needs to be able to know what counties/cities the stations are in and be able to accept some sort of plain text/string input (Job of the below task). - This needs to able to function if multiple files are pulled for a specific location or if none are pulled for a specific location This final task may also be a reason that NOAA data may not be useable
Geo-Location	This needs to be a one time running task that takes isd-history-csv and not only cleans it up to be bare minimum data to save space and allow downloading of files to work, but needs to be able to be called easily. Alternatively a single csv file (of small enough) could be saved to the github This works, but needs to have the file be cleaned and shrunk down a bit. I would love some start up function in the case that isd-history needs to be updated, but this can be a later task

Geo-Location

Using the above table, the best move is to start from the bottom with Geo-Location. This is mostly done, but deciding on a singular file and singular function that can be run is good in case the file needs to be replicated at any time.

The file isd-history.csv or integrated surface database history provides the following fields:

```
"USAF","WBAN","STATION NAME","CTRY","STATE","ICAO","LAT","LON","ELEV(M)","BEGIN","END"
```

While I was testing feasibility, I had Cursor generate a way to get the city and county that each of the stations resided in. It would take the LAT and LON of the isd-history, translate that into X and Y coords for the census to get the city and county locations.

This would turn into isd-history-gecoded.csv with columns like so.

```
USAF,WBAN,STATION_NAME,CTRY,STATE,ICAO,LAT,LON,ELEV(M),BEGIN,END,Geocoded_City,Geocoded_C
ounty
```

This step is complete and was complete from previous work before I had gotten confirmation. The next step is to clean up the files used to generate all the information I needed and then start a 'master training data' file with this as an optional first step.

After working with Cursor and other AI tools, csv_geocode.py is now fully operational. A master file is also available called CreateTrainingData.py though it may be renamed at some point soon to more closely match with current naming scheme. geocode(), the function imported by csv_geocode.py takes a src and dst location as well as a state and ctry (country). These need to be obtained from isd-history.csv to ensure that the spelling is correct but allows for specific counties and states to have their information overseen. Given that this project seems to value Texas over anywhere else, the defaults have been set to ctry="US" state="TX" and the saved geocode.csv which will be saved to the github, holds the stations with this information.

The main function can be run by:

```
py csv/CreateTrainingData.py --geocode
```

From here, there is likely some information that can be reduced to make the file smaller. While this already is not a large file, given that it is to be saved to github it could be smaller. Some of the columns like USAF, WBAN, and some of the ID information could be removed. However, given that this information is to feed the download file portion of this task, it would be better to remove columns there.

Downloading of files

Now that geocoded.csv exists, the code has a shortlist of stations to download information from. However, there is still the issue that some stations exist that may not have a csv file lined up for them. There are also multiple stations per county and in some cases per city. Thus, the following things must be looked into:

- How many counties are actually represented with csv data
- What to do if a csv file does not exist per city/county
- What to do if multiple fields exist per city/county

Given that only the bare bones of this function has been completed, the first thing that needs to be done is to get the functionality down. This functionality will be located inside of weather_download.py.

For this file, we look into this url: https://www.ncei.noaa.gov/data/global-hourly/access/

Year can be appended to the end to open specific directories and then from there, specific stations can be chosen from. For example, the below is the DFW airport that the original preprocessing was built off.

https://www.ncei.noaa.gov/data/global-hourly/access/2025/72258013960.csv

Simply clicking on it will download it. From here, we can place this into a csv folder inside of the weather folder. From there, preprocessing can be applied to each file.

After running the downloader to get all the county information, I saw there were a lot of repeated stations, even beyond that there were a lot of repeated counties. Unfortunately, this step has reduced the amount of counties that are available to sample:

```
Failed Counties:  16
Failed Counties List:  ['Wilbarger County', 'Red River County', 'Fannin County',
'Montague County', 'Andrews County', 'Willacy County', 'Washington County', 'Fayette
```

```
County', 'Kinney County', 'Shackelford County', 'Liberty County', 'Kent County', 'Palo
Pinto County', 'Maverick County', 'Hudspeth County', 'Donley County']

Success Counties:  130
Success Counties List:  ['Llano County', 'Brewster County', 'Wise County', 'Brooks
County', 'Howard County', 'Matagorda County', 'Hidalgo County', 'Hood County', 'Grayson
County', 'Dallas County', 'Hill County', 'Jasper County', 'Cherokee County', 'Ellis
County', 'Robertson County', 'Titus County', 'Gray County', 'Anderson County', 'Ochiltree
County', 'Nueces County', 'Young County', 'Johnson County', 'Gillespie County', 'Bee
County', 'Deaf Smith County', 'Atascosa County', 'Wichita County', 'Travis County', 'Rusk
County', 'Stephens County', 'Zapata County', 'Harris County', 'Waller County', 'Roberts
County', 'Fort Bend County', 'Lampasas County', 'Crockett County', nan, 'Dimmit County',
'Coleman County', 'Limestone County', 'Panola County', 'Jefferson County', 'Harrison
County', 'Taylor County', 'Milam County', 'Henderson County', 'Rockwall County',
'Presidio County', 'Wood County', 'Tarrant County', 'Gaines County', 'Upshur County',
'Lee County', 'Scurry County', 'Reeves County', 'Hale County', 'Nolan County', 'Uvalde
County', 'Calhoun County', 'Moore County', 'Hamilton County', 'Coryell County', 'Midland
County', 'Jim Hogg County', 'Wharton County', 'Hopkins County', 'Edwards County', 'San
Patricio County', 'Guadalupe County', 'Galveston County', 'Brazoria County', 'Montgomery
County', 'Angelina County', 'Walker County', 'Smith County', 'Navarro County', 'Gregg
County', 'Kaufman County', 'Nacogdoches County', 'Cameron County', 'Kleberg County', 'Jim
Wells County', 'Webb County', 'Bexar County', 'Aransas County', 'La Salle County',
'Medina County', 'Kerr County', 'Caldwell County', 'Collin County', 'Burnet County',
'Williamson County', 'Victoria County', 'Cooke County', 'McLennan County', 'Bell County',
'Lamar County', 'Hunt County', 'Denton County', 'Parker County', 'Erath County', 'Val
Verde County', 'Terrell County', 'Pecos County', 'Culberson County', 'Tom Green County',
'Hemphill County', 'Ector County', 'Winkler County', 'Brown County', 'Lubbock County',
'El Paso County', 'McCulloch County', 'Orange County', 'Potter County', 'Hutchinson
County', 'Childress County', 'Burleson County', 'Houston County', 'Sutton County',
'Comanche County', 'Hartley County', 'Kimble County', 'Brazos County', 'Kenedy County',
'Gonzales County', 'Ward County', 'Bailey County', 'Coke County']
```

This brings the 145 counties down to 130 (though somehow an extra county snuck into the failed counties. After looking I still have no idea where that came from). I pulled the entire list into documentation as I did have one issue where I had made an error and there were some duplicates. I have checked this list and there shouldn't be any duplicates in this list.

From here, the basic function now works. The following tasks still need to be completed:

- Some list of counties to explore. ex: ["Dallas", "Fort Worth"] and then getting the results of only those counties
- The ability to toggle between getting one station per county and one station per city
- Renaming the files after the respective city.county

The **first** is extremely easy to get done and is only a slight alteration of how the code already runs.

The **second** requirement is also fairly straightforward and sets up the third requirement. To do the testing for this requirement as well as the last one, I will be using Dallas County which is the county we are using in the preliminary testing data regardless.

When just Dallas is being looked for inside of the CreateTrainingData.py and cities (a boolean used to control if cities are being search for in addition to counties are being searched for) is false, we get this output

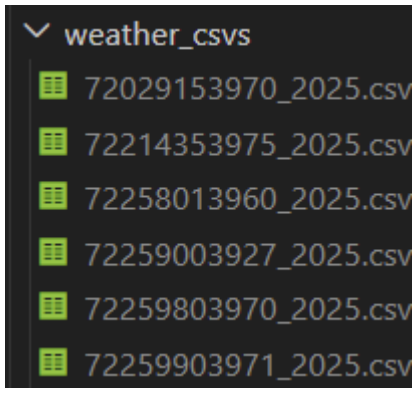
```
Successfully downloaded weather data for 1 counties
```

Meanwhile, if cities is set to true, this becomes the output.

```
Successfully downloaded weather data for 6 cities
```

6 cities should provide a noticeable difference. There is an issue that will need to be addressed later though, there are more than 6 cities in the county of Dallas. It has never been feasible to get all the city information for Dallas, but this means that, even if just for Dallas, we need some sort of default behavior for when a county/city does not have weather that reflects its location. While this could be an issue for preprocessing, the third issue may also be able to reflect a solution for preprocessing.

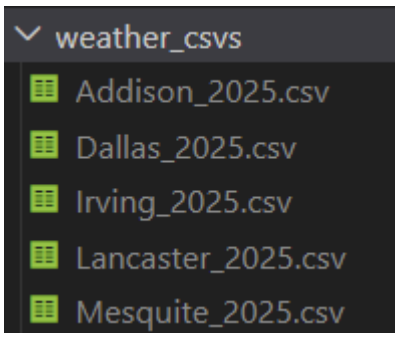
The **third** requirement needs to address naming. At present the downloaded files have this naming schema:



Ideally, the files should be named after the city they are taken from if cities (the flag that allows the file to look for cities as opposed to just counties) or the counties if cities are false. Given that multiple years are taken, the year should be added to the end. So we should get something like:

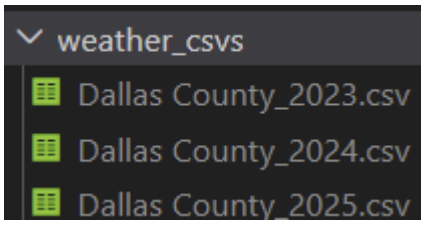
- Dallas_2025.csv
- FortWorth_2025.csv
- Dallas_2024.csv
- Ect.

After alteration, weather_csvs has these files



Once again, I'm not sure why the amount of cities has changed, but it apparently has. Regardless, 5 cities is still decent for this experiment. While 30+ cities would provide a more robust test, ultimately this is supposed to sum the 130 counties it was able to locate so the fact that not all Dallas Cities are accounted for is not an issue. However, for a larger test at some point, we could combine DFW as well as Dallas County. This may require that both city name and county name are placed together to allow better city/county overlap for crash records. However, this can be addressed much later.

I moved back to cities = false and just Dallas County to show that multiple years work just fine.



From here, I want to clean up geocoded.csv. USAF + WBAN make up the station id. Station Name is helpful for human readability but is otherwise unimportant. CTRY and STATE are mandatory. ICAO is unknown to me but does not seem to contribute to anything. LAT and LON are necessary for geocoding, but afterwards do not provide any value so can be removed after geocoding is done. ELEV(M), BEGIN, and END have also had no use to the process at any given point so they too can be removed. The new saved geocoded.csv has these columns and I can confirm that the whole pipeline works up to where I am now.

```
USAF,WBAN,STATION NAME,CTRY,STATE,Geocoded_City,Geocoded_County
```

Missing location

I did bring up the concept of the missing cities/counties as they correlate to the data. My first thought when I saw that we have only 145 counties out of 254 was that for the scope of this project, the other 109 could be ignored given that they were probably more rural. The model will provide some context even to counties/cities that do not contribute history through things like Rural Urban Tag, Weather, and perhaps light level down the line. Weather is needed to create negative samples so any locations that do not have negative samples would need to be removed.

Given that the whole process of automating weather was designed to provide better scope of data, this seems like a poor idea for the second draft of training data. So, my next thought was to use a ‘default location.’ Where any city that wasn’t represented would get its weather from. I figured because our prototype data was using Dallas and the final scope of all of Texas would use Dallas, I could make the Dallas file a Default.csv where any location that does not have a corresponding csv file inside weather_csvs could pull from. The problem is, this means that only 4 locations, or 4/30 of the 3 million rows change which is insignificant at best. I feel as if we can still do better when matching up crash data and locations to weather.

The third idea I had was to do more geolocation/caching. Especially for city based weather (and even scaling up to counties potentially), we could use some level of geolocation to find what station/city with weather is closest to one without and then use that location’s weather instead. This still leaves some incorrect weather assumptions, but would theoretically improve weather coverage across the board.

However, I am writing this at 3:26 am on 4/3/2026 and thus I am running out of time and energy to complete the preprocessing and then the data generation phase of this step so for now I will simplify this process. I will move on to preprocessing. Once I get to data generation I will reassess where I am and make the decision for what I want to show off at the meeting. However, I want to use the geolocation idea in the final TrainingData as I believe that will yield results most closely associated with reality which is the end goal of the project.

Preprocessing

4/2/2026 Meeting

My questions about if 145 counties will work for the TrainingData so long as I can get good data for Dallas County which I believe I can. I have also been told that I can and should focus on data generation which is good as this is taking much longer than I thought would.

The idea of adding light levels to the training data was also brought up. I do like this idea and I think it would be good to implement. I think the main problem would be how we would be able to get light levels in real time. While daytime meaning high light level is easy, I think at night it would be hard to distinguish between lighted roads and unlighted roads.

My idea to generate negative samples is to take roads that have shown to be lit at night and then just save those to a file for later reuse seems to be the same solution to solve the problem of the app figuring out what roads are lit in real time. While this is feasible, I do worry about accuracy in case of power outages or storms or other events.

Regardless, In the coming weeks I will address this and attempt to add it into the training data. I think I would like to get the weather done and then get the large entire year of Texas crash records converted before I begin working on

the light level. The giant crash record file has different column names and I want to be sure I can rebuild what already exists before moving on to adding new things to the data.

In terms of the model, last week on 3/27/2026, the idea of retraining the model on user submitted crashes was brought up. I mentioned during this time the limited weather constraints in that we only had to about 08/19/2025 (month/day/year) in terms of weather. I don't know if just adding negative samples to the model and then training on that and just negative samples would be wise. Until more weather is updated, it would be impossible to train the model on new negative samples. However, given the current ratio of positive to negative samples, there is a chance that as long as the user does not send the year to the AI model, things could balance themselves out. Additionally when NOAA updates their weather, the model could be trained on more stable data.

Next Week's Tasks

- Finish The Weather Automation System
    - Map Cities/Counties without a station to the nearest station that works
    - Finish Preprocessing
    - Adjust data creation to reflect the new system
- Begin working on converting the yearly C.R.I.S. data to become useable data
    - Look for light level and begin working on adding in that data
    - Remove the Rural Urban Flag